

65. Server Sent Events (EventSource)

Built-in class `EventSource` server se connection banaye rakhti hai aur server se events lene deti hai. `WebSocket` ki tarah persistent, par kuch farak hain:

<code>WebSocket</code>	<code>EventSource</code>
Bi-directional (dono bhej sakte hain)	One-directional (sirf server bhejta hai)
Binary aur text	Sirf text
WebSocket protocol	Regular HTTP

`EventSource` kam powerful hai, par **simple** hai. Kai apps me WebSocket ki taqat zaroorat se zyaada hoti hai. Agar bas server se data ka stream chahiye (chat messages, market prices), to ye kaafi hai. Saath me **auto-reconnect** built-in hai (WebSocket me khud likhna padta hai), aur ye purana simple HTTP hi hai.

Messages lena

```
js
let eventSource = new EventSource("/events/subscribe");

eventSource.onmessage = function(event) {
  console.log("New message", event.data);
};
// ya eventSource.addEventListener('message', ...)
```

Browser `url` se connect ho ke connection khula rakhta hai.

Server ko:

- Status **200** aur header `Content-Type: text/event-stream` dena hai
- Connection khula rakhke messages is special format me likhne hain:

```
data: Message 1

data: Message 2

data: Message 3
data: of two lines
```

- Message ka text `data:` ke baad (colon ke baad space optional)

- Messages `\n\n` (double line break) se alag
- Line break bhejne ke liye ek aur `data:` line
- Practically complex messages JSON me bhejte hain (`data: {"user":"John","message":"First line\n Second line"}`), to ek `data:` = ek message maan sakte hain.

Cross-origin

`fetch` jaisa hi. Remote server `Origin` header dekhta hai aur `Access-Control-Allow-Origin` bhejta hai. Credentials chahiye to:

```
js
new EventSource("https://another-site.com/events", { withCredentials: true });
```

Reconnection

Connection toota to **khud reconnect** hota hai (kuch second ke delay ke baad). Server delay suggest kar sakta hai (ms me):

```
retry: 15000
data: Hello, I set the reconnection delay to 15 seconds
```

- Server chahta hai ki browser reconnect **na** kare: HTTP status **204**.
- Browser khud band karna chahe: `eventSource.close()`.
- Galat `Content-Type` ho ya HTTP status 301, 307, 200, 204 ke alawa ho to reconnect nahi hota, `error` event aata hai.
- Band ho gaya connection dobara "reopen" nahi hota, naya `EventSource` banao.

Message id

Connection toot jaye to pata nahi kaun se messages mile aur kaun se nahi. Sahi resume ke liye har message me `id`:

```
data: Message 1
id: 1

data: Message 2
id: 2

data: Message 3
data: of two lines
id: 3
```

`id:` wala message milne par browser:

- `eventSource.lastEventId` set karta hai
- Reconnect par header `Last-Event-ID` bhejta hai taaki server aage ke messages dobara bhej sake

`id:` ko `data:` ke **neeche** rakho (taaki message milne ke baad hi `lastEventId` update ho).

Connection status: `readyState`

- `EventSource.CONNECTING = 0`: connect ya reconnect ho raha
- `EventSource.OPEN = 1`
- `EventSource.CLOSED = 2`

Event types

Default 3 events:

- `message`: message mila (`event.data`)
- `open`: connection khul gaya
- `error`: connection nahi ban paaya (jaise server ne 500 diya)

Server alag event type de sakta hai, `event: ...` se (`data:` se pehle):

```
event: join
data: Bob

data: Hello

event: leave
data: Bob
```

Custom events ke liye `addEventListener` (`onmessage` nahi):

```
js

eventSource.addEventListener('join', event => alert(`Joined ${event.data}`));
eventSource.addEventListener('message', event => alert(`Said: ${event.data}`));
eventSource.addEventListener('leave', event => alert(`Left ${event.data}`));
```

Full example (server ka idea)

Server 1, 2, 3 bhejta hai, phir `event: bye` aur connection tod deta hai. Browser apne aap phir connect ho jaata hai.

```
js
```

```
res.writeHead(200, {
  'Content-Type': 'text/event-stream; charset=utf-8',
  'Cache-Control': 'no-cache'
});
res.write('data: ' + i + '\n\n');
res.write('event: bye\n');
res.write('data: bye-bye\n\n');
```

Client me `onopen`, `onerror` (jisme `readyState == EventSource.CONNECTING` check karke pata chalta hai reconnect ho raha hai ya nahi), `onmessage` aur `addEventListener('bye', ...)`).

Summary

`EventSource` khud persistent connection banata hai aur server ko messages bhejne deta hai. Fayde:

- **Auto reconnect** (`retry` timeout tunable)
- Resume ke liye **message ids** (`Last-Event-ID`)
- `readyState` se state

Isliye WebSocket ka viable alternative hai (WebSocket low-level hai aur ye features nahi deta, unhe khud banana padta hai). Kai real apps me itna kaafi hota hai. Saare modern browsers me (IE me nahi).

js

```
let source = new EventSource(url, [credentials]); // doosra argument: { withC
```

Properties: `readyState`, `lastEventId`. **Method:** `close()`. **Events:** `message`, `open`, `error`.

Server response format

Messages `\n\n` se alag. Fields:

- `data:` message body (kai `data:` = ek message, beech me `\n`)
- `id:` `lastEventId` update karta hai (usually aakhri field)
- `retry:` reconnection delay (ms). JS se set nahi kar sakte.
- `event:` event ka naam (`data:` se pehle)